

Different Types of Errors Exit in 'C' ✖

These errors are detected either during the time of compilation or execution. Thus, the errors must be removed from the program for the successful execution of the program.

There are mainly five types of errors exist in C programming:



Syntax error

Errors in code structure



Run-time error

Errors during execution



Linker error

Errors in linking files





Logical error

Incorrect program logic



Semantic error

Meaningful but incorrect statements



Syntax Error

Syntax errors are also known as the compilation errors as they occurred at the compilation time, or we can say that the syntax errors are thrown by compilers. These errors are mainly occurred due to the mistakes while typing or do not follow the syntax of the specified programming language. These mistakes are generally made by beginners only because they are new to the language. These errors can be easily debugged or corrected.



Common Syntax Errors:



If miss the parenthesis (}) while writing the code.



Displaying the value of a variable without its declaration.



If miss the semicolon (;) at the end of the statement.



Example 1:

```
#include <stdio.h>

int main()
{
    a = 10; // variable 'a' not declared
    printf("The value of a is : %d", a);
```

```
return 0;  
}
```



Example 2:

```
#include <stdio.h>  
  
int main()  
{  
    a = 2;  
    if(.) // syntax error  
    printf("a is greater than 1");  
    return 0;  
}
```



2/6

⚡ **Run-time Error**

Sometimes the errors exist during the execution-time even after successful compilation known as runtime errors. When the program is running, and it is not able to perform the operation is the main cause of the run-time error. The division by zero is the common example of the run-time error. These errors are very difficult to find, as the compiler does not point to these errors.



Example:

```
#include <stdio.h>

int main()
{
    int a=2;
    int b=2/0; // division by zero
    printf("The value of b is : %d", b);
    return 0;
}
```



Common Causes

Division by zero, null pointer dereference, array out of bounds

Detection

Difficult to detect as compiler doesn't point to these errors

Solution

Debuggers, error handling, input validation

3/6

Linker Error

Linker errors are mainly generated when the executable file of the program is not created. This can be happened either due to the wrong function prototyping or usage of the wrong header file.

For example, the main.c file contains the sub() function whose declaration and definition is done in some other file such as func.c. During the compilation, the compiler finds the sub() function in

func.c file, so it generates two object files, i.e., main.o and func.o. At the execution time, if the definition of sub() function is not found in the func.o file, then the linker error will be thrown.

The most common linker error that occurs is that we use Main() instead of main().



Example:

```
#include <stdio.h>

int Main() // should be main() not Main()
{
    int a=78;
    printf("The value of a is : %d", a);
    return 0;
}
```



Common Causes

Wrong function names,
missing function definitions,
incorrect header files

Detection

Detected during linking
phase after compilation

Solution

Check function names,
include proper header files,
verify function definitions

4/6

Logical Error

The logical error is an error that leads to an undesired output. These errors produce incorrect output, but they are error-free, known as logical errors. These types of mistakes are mainly done by beginners. The occurrence of these errors mainly depends upon the logical thinking of the developer. If the programmers sound logically good, then there will be fewer chances of these errors.



Example:

```
#include <stdio.h>

int main()
{
    int sum=0; // variable initialization
    int k=1;
    for(int i=1;i<=10;i++); // logical error, as we put the semicolon after loop
    {
        sum=sum+k;
        k++;
    }
    printf("The value of sum is %d", sum);
    return 0;
}
```



Common Causes

Incorrect loop conditions,
wrong calculations,



Detection

Difficult to detect as code
compiles and runs without
crashing



Solution

Code reviews, step-by-step
debugging, test with known
inputs



incorrect conditional
statements

5/6

Semantic Error

Semantic errors are errors that occurred when the statements are not understandable by the compiler.



Common Cases for Semantic Error:



Use of a un-initialized variable

```
int i;  
i=i+2;
```



Type compatibility

```
int b = "Error Handling";
```



Errors in expressions

```
int a, b, c;  
a+b = c;
```



Array index out of bound

```
int a[10];  
a[10] = 34;
```



Example:

```
#include <stdio.h>  
  
int main()  
{
```

```
int a,b,c;  
a=2;  
b=3;  
c=1;  
a+b=c; // semantic error  
return 0;  
}
```

Key Takeaway

Understanding these different types of errors is crucial for effective debugging and writing robust C programs. Each error type requires different detection and resolution strategies.